

Python API 文档

1. Database API

1.1. 数据库连接建立与关闭

1.1.1. DatabaseConn：建立数据库连接

- param:
 - jdbc_http_proxy: timechord服务对应的ip:port, 可以在manager的timechord配置项页面的timechord.server.port上找到对应值
 - real_conn: 实际quark服务的ip:port
 - db: 指定访问的database
 - auth_type: 指定安全登录方式(默认为simple, 不开启安全登录)
 - simple: 支持
 - ldap: 支持, 需要指定username,password,guardianToken
 - guardian: 支持, 需要指定guardianToken, token信息可以从guardian server获取
 - kerberos: 暂不支持
 - compressSQL: sql是否压缩传输(默认为false, sql不压缩)
 - session_timeout: 当前session的超时时间（即无操作后经过此时间session会被关闭），单位ms，默认5分钟
 - 注：实际被关闭的时间会有随机的延长，取决于服务内部的检查间隔（默认50s）
 - auto_close: 是否在执行结束后自动关闭当前session。默认是True，每次执行接口后会关闭当前session
 - 以下接口不受此参数影响，默认执行结束不关闭当前session：
 - run_simple_command、run_set_command、set_crux_mode、set_quark_mode、close_connection
 - login_timeout: 建立session时的超时时间，超过此时间未成功建立session，会报出timeout提示。单位ms，默认100s
 - real_ui_host_port: 默认为None。支持本地读时需要填写此参数，值为quark server的ip:quark的inceptor.ui.port
 - disable_cancel: 默认为False。为True时取消自动cancel功能（多线程使用DatabaseConn时需要）
 - run_mode: sql执行方式。默认值为"crux"
 - "quark": 支持直接sql执行
 - run_tdc: 指定是否在tdc上执行
 - tdc_quark_ui_port: 指定tdc上quark的ui端口, 这里的端口为tdc上对应quark的http外部端口
- simple方式建立连接

```
conn = DatabaseConn("ip:port", "jdbc:hive2://ip:port", "database_name")
```

- ldap方式建立连接

```
conn = DatabaseConn("ip:port", "jdbc:hive2://ip:port", "database_name",
                    auth_type="ldap",
                    token="MA2pQMi8pof7qsYoxpIK-TDH",
                    username="test_user",
                    password="66666666")
```

- guardianToken方式建立连接

```
conn = DatabaseConn("ip:port", "jdbc:hive2://ip:port", "database_name",
                    auth_type="guardian",
                    token="MA2pQMi8pof7qsYoxpIK-TDH")
```

- tdc场景建立连接

```
conn = DatabaseConn("ip:port", "jdbc:hive2://ip:port", "database_name",
                    run_mode="quark",
                    run_tdc=True, tdc_quark_ui_port="30602",
                    auth_type="guardian",
                    token="c2buNBTb9cVUM6jU6sb7-TDHTEST.TDH")
```

- 设置session默认超时时间

```
conn = DatabaseConn("ip:port", "jdbc:hive2://ip:port", "database_name",
                    session_timeout=60000) # 单位:ms
```

- 设置建立session时的超时时间

```
conn = DatabaseConn("ip:port", "jdbc:hive2://ip:port", "database_name",
                    login_timeout=100000) # 单位:ms
```

1.1.2. 关闭数据库连接

```
conn = DatabaseConn(...)
conn.close_connection()
```

1.2. 元数据访问函数

元数据访问函数通过已建立的DatabaseConn执行

1.2.1. show_databases()

显示所有库的库名，返回值为list

```
conn.show_databases()
```

1.2.2. show_tables()

显示当前database下的所有表，返回值为list

```
conn.show_tables()
```

1.2.3. show_columns(tblName)

显示指定表的schema信息, 返回值为list

- param:
 - tblName: 表名

```
conn.show_columns("tbl1")
```

1.2.4. show_properties(tblName)

返回指定表的properties。如果需查询mapping表的信息，需传入真实的表名。

- param
 - tblName: 表名

```
conn.show_table_type("tbl1")
```

1.2.5. show_table_type(tblName)

返回指定表的类型，目前有"timelyre"、"orc"、"timelyre2"和"holodesk"，其它表返回"unknown"

- param
 - tblName: 表名

```
conn.show_table_type("tbl1")
```

1.3. 表操作函数

表操作函数通过已建立的DatabaseConn执行

1.3.1. drop_table(tblName)

删除指定表

- param:
 - tblName: 表名

```
conn.drop_table("tbl1")
```

1.3.2. truncate_table(tblName)

清空指定表的数据

- param:
 - tblName: 表名

```
conn.truncate_table("tbl1")
```

1.3.3. create_table(tblName, tblType, schema, tags, timecol, ...)

创建一张表

- param
 - tblName: 表名
 - tblType: 表的类型, 支持timelyre, orc, text类型。使用以下值指定:
 - "timelyre": 需要指定如下参数
 - tags: 必填, 指定tag列
 - timecol: 必填, 指定时间列
 - shard_group_duration: 必填, 指定分片内时间跨度, 单位是s。可以设置为31536000 (1年)。金融tick级数据推荐604800(7天), 日频数据推荐157680000(5年)
 - uniqcols: 可选, 指定rowkey列
 - properties: 可选, 其他指定的建表参数
 - "orc":
 - bucketcolumn: 指定分桶键
 - bucketquantity: 指定分桶数
 - "text": 可以指定如下参数
 - isExternal: 是否为外表
 - col_delimiter: 指定每行中列之间的分隔符
 - location: 指定实际数据在hdfs上的目录
 - schema: list 指定表的列名和对应类型
- timelyre建表示例

```
conn.create_table(tblName="tbl_timelyre", tblType="timelyre",
                  schema=[("a", "int"), ("b", "string"), ("c",
"timestamp")],
                  tags='b',
                  timecol='c',
                  shard_group_duration='31536000')
```

- orc建表示例

```
conn.create_table(tblName="tbl_orc", tblType="orc",
                  schema=[("a", "int"), ("b", "string"), ("c",
"timestamp")],
                  bucketcolumn='a',
                  bucketquantity='3')
```

- text建表示例

```
conn.create_table(tblName="tbl_text", tblType="text",
                  schema=[("a", "int"), ("b", "string"), ("c",
"timestamp")],
                  isExternal=True,
                  col_delimiter=',',
                  location='/tmp/python_test/data')
```

1.3.4. insert(table, values, batch_size, parallelism)

- param
 - table: 表名
 - values: 插入的数据, 支持DataFrame或者list数据的形式
 - batch_size: 内部插入数据每批的大小, 默认为2000
 - parallelism: 并发度, 默认为1

```
conn.insert("tbl1", values)
```

1.3.5. upload_df(df, schema, table_name, batch_size, use_parallel, max_worker_num)

将pandas.DataFrame格式的数据上传至外表（外表名以"temp_df_table_"开头）。若指定目标表，则会将数据从外表导入目标表。返回值为：外表名，外表的hdfs_path

- param
 - df: 必填，待上传的数据，格式为pandas.DataFrame
 - schema: 必填，外表的schema

- `table_name`: 可选, 上传的目标表表名。如果指定目标表, 则会将数据从外表导入目标表, 并在最后自动清理外表数据。默认值为`None`, 即不指定。
 - `database`: 可选, 上传的目标表的库名, 默认值为当前数据库连接的库名
 - `batch_size`: 可选, 每批处理数据的行数, 默认值为200000
 - `use_parallel`: 可选, 是否并发上传数据, 默认值为`True`
 - `max_worker_num`: 可选, 上传数据的最大并发量, `use_parallel`为`True`时生效, 默认值为30
- 仅将数据上传至外表, 不会自动进行外表数据清理

```
import pandas as pd
schema = [['task_id', 'string'],
          ['text_value', 'string'],
          ['datetime', 'timestamp']]

df = pd.DataFrame({
    'task_id': ["xxx"],
    'text_value': ["测试值2"],
    'datetime': ["2020-01-01 00:00:00"],
})

# 将数据上传至外表
ext_tbl, hdfs_path = conn.upload_df(df, schema)

# 手动清理外表和hdfs数据 (可选)
conn.run_simple_command(f"hdfs -rm -r {hdfs_path}")
conn.drop_table(ext_tbl)
```

- 将数据上传至外表并导入目标表, 导入完成后会自动清理外表数据

```
import pandas as pd
schema = [['task_id', 'string'],
          ['text_value', 'string'],
          ['datetime', 'timestamp']]

df = pd.DataFrame({
    'task_id': ["xxx"],
    'text_value': ["测试值2"],
    'datetime': ["2020-01-01 00:00:00"],
})

table_name = "dst_table"

# 将数据上传至目标表。会自动清理外表和hdfs数据
conn.upload_df(df, schema, table_name)
```

1.3.6. query_raw_data(sql)

指定sql获取数据。返回List类型的数据

- param
 - sql: 直接查询的sql语句

```
conn.query_raw_data(sql="select * from tbl limit 10")
```

1.3.7. query_as_df(tbl, query, select_cols, condition, combine_ignore_index, time_range_hint, local_dir)

从指定表里按照指定条件查询相应数据, 并返回pd.DataFrame类型的数据。性能较优

- param
 - tbl: 表名
 - query: 直接查询的语句(query不为空,则直接执行query语句)
 - select_cols: (可选)指定查询的列名
 - condition: (可选)指定查询的条件, 采用SQL where条件里的写法
 - combine_ignore_index: (可选)query的结果分为多个文件, 决定多个文件的df合并为一个时的选项
 - time_range_hint: (可选)当查询中指定order by time的场景下, 增加改参数可以提升查询性能, 输入为一个字符串, 用逗号分割, 两个数字分别表示查询数据范围内的最小时间戳和最大时间戳
 - local_dir: (可选)设置查询结果文件在本地存放的目录。默认值为None。此参数为默认值时函数返回DataFrame, 而不是存储在本地。

```
conn.query_as_df("tbl1", query="select * from tbl where code = '1.CN' and  
datetime >= '2020-12-21 00:00:00' and datetime < '2020-12-22 00:00:00'")
```

```
conn.query_as_df("tbl1", select_cols="code,timedeate,price1",  
condition="code = '1.CN'", combine_ignore_index=True)
```

```
conn.query_as_df("tbl1", query=f"select * from default.tbl1 where datetime  
>= '2018-12-22 08:00:00' and datetime < '2021-12-31 00:00:00' order by  
datetime", time_range_hint="1609718400000, 1648771200000")
```

1.3.8. direct_query_as_df(table_name, cols, tag_value_list, time_filter)

读取指定表的数据, 返回dataframe。使用此函数要求建立DatabaseConn时填写正确的real_ui_host_port参数。

- param
 - table_name: str
 - 必选参数, 表名
 - cols: List[str]
 - 可选参数, 默认值为None, 需要选择的列名, 比如['code', 'ts', 'high']
 - tag_value_list: list[str]
 - 可选参数, 默认值为None, 需要选择的tag列表, 比如['100.SH', '200.SZ']
 - time_filter: list[str]

- 可选参数，默认值为None，是一个长度为2的list，分别是时间戳filter的上下界,比如 ['2020-01-01 08:00:00.1','2020-01-02 09:00:00.0']

```
table_name = "test"
cols = ['code', 'ts', 'high']
tag_value_list = [f"{i}.CN" for i in range(10, 100)]
time_filter = ['2010-01-01 08:00:00', '2010-01-30 08:00:00']
df = conn.direct_query_as_df(table_name, cols=cols,
tag_value_list=tag_value_list, time_filter=time_filter)
```

其它例子可以参考demo/local_read目录下的demo文件

1.4. 其他操作函数

其他操作函数通过已建立的DatabaseConn执行

1.4.1. show_session_state()

返回当前session状态：最大可建立session数，当前已建立session数

```
conn.show_session_state()
```

1.4.2. close_inactive_sessions(period)

关闭非活跃时间超过指定时间的所有session

- param
 - period: 非活跃时间长度，单位ms

```
conn.close_inactive_sessions(60000) # 关闭非活跃时间超过1分钟的所有session
```

1.4.3. set_session_name(sn, basic)

建立DatabaseConn连接时，会使用基础的session name

可以通过此接口设置新的session名称。可选是否在名称后附加基础信息，默认不附加

- param:
 - sn: session名称
 - basic: 指定设置新的session名称时是否附带basic信息，默认为False

```
conn.set_session_name("myname")
```


- 效果示例：
 - 基础session name包含以下信息：
 - USER：用户名，默认为anonymous
 - IP：当前机器ip地址
 - PID：当前进程id
 - AC：是否开启auto_close
 - FILE：当前进程执行的python脚本名。如果使用jupyter等交互形式，将无此项信息

```
# 基础session name示例：  
[USER]:anonymous, [IP]:172.16.182.166, [PID]:366888, [AC]:True,  
[FILE]:/home/transwarp/my_test.py
```

- 调用set_session_name后，session name为：

```
conn.set_session_name("myname")  
# 设置后的session name：  
# myname  
  
conn.set_session_name("myname", basic=True)  
# 设置后的session name：  
# myname, [USER]:anonymous, [IP]:172.16.182.166, [PID]:366888,  
[AC]:True, [FILE]:/home/transwarp/my_test.py
```